

Marcos Javier Alvarez

Senior Full-Stack Engineer · AI/LLM Systems in Production

Cúcuta, Colombia (Remote) · garusis@gmail.com · github.com/garusis · linkedin.com/in/garusis

Portfolio & detailed career evidence: hire-me-mcp-web.vercel.app · **Query it from any MCP client:**
<https://hire-me-mcp-web.vercel.app/api/mcp>

Summary

Senior full-stack engineer specialized in taking AI from demo to production. Over the past four years I have built LLM pipelines for document data extraction, communication infrastructure that syncs and classifies email and SMS at scale, and agentic workflows that are tested, cost-traced, and measured like any other engineering system. What I care about most is the gap between an AI demo and an AI system: evaluation harnesses, human-graded ground truth, cost discipline, and observability. Full-stack range across event-driven backends, React frontends, and infrastructure.

Experience

House Numbers — Senior Full Stack Engineer · May 2022 – Present

Core engineer on a 15-service TypeScript monorepo for an AI-powered mortgage-lending platform, working across backend services, frontend, and infrastructure.

- Led a multi-round proof of concept evaluating vision-LLM document splitting, classification, and extraction as a candidate replacement for the incumbent OCR vendor, including the human-reviewed corpus, evaluation and cost tracing, and fixes to measurement bugs in the scoring. The PoC stayed experimental: production kept the vendor plus the existing LLM fallback, and productionization was left as a later team decision.
- Built and still owns the platform's communication service end to end: asynchronous, webhook-driven email and SMS/call intake, LLM-assisted message classification kept separate from the deterministic rules that route each message, and loan matching over field-level-encrypted data.
- Turned agentic AI features into engineered systems: evaluation harnesses with LLM-as-judge scoring and human-graded ground truth, per-call cost tracing, version-controlled prompts, and an AI cost-reduction workstream.
- Built and evolved flexible query filtering for the loan-application product end to end: the frontend filter flows and a reusable TypeScript filter library and query parser that compose MongoDB queries from them — multi-value, related-entity, date-range, set/unset, and regex-escaped text-search operators — with tenant-scoped access for sensitive loan data.
- Designed and delivered a monorepo-wide, agent-consumable documentation system with deterministic integrity gates in CI, alongside broad ownership of shared packages, public upload flows, user offboarding, observability, and production incident response.

Tech: TypeScript, Node.js, React, MongoDB, AWS, Redis, Kubernetes, OpenAI API, Anthropic API.

Returning from paternity leave into an AI-driven company pivot

Situation: When I returned to House Numbers after a long paternity leave, the engineering team consisted of three people and the company still operated with a B2C model. Two weeks later, our CEO told us that the model was not working and that the company would pivot to a B2B mortgage platform built heavily around AI. This was early in the LLM boom. We had run a few experiments, but none of us had a real background in machine learning, LLMs, or agents. I normally have a lot of confidence in my ability to learn and solve technical problems. This time I could not see it. I was exhausted, emotionally overwhelmed, and still adapting to becoming a father when the company changed both its business and its technical direction. The combination made the new work feel beyond me.

Task: I seriously considered leaving because I did not want to slow down the team or become a burden while everyone else was trying to make the pivot work. I had to decide whether to stay and learn the new AI work or leave.

Actions:

- Before making that decision, I spoke openly with my CTO, who told me that the rest of the team was probably facing the same technical uncertainty. A teammate then shared that he also felt a strong sense of impostor syndrome around the new AI work.
- My wife and my colleagues could still see my ability clearly during a period when I could not. They listened and adjusted expectations. Some of the workload was shared, and I had space to recover while learning alongside them.
- Once I started working directly with LLMs, the change in perspective came quickly. The names and tools were new, but the underlying work still involved problems I had handled throughout my career: system boundaries, data contracts, failure modes, testing, and operational tradeoffs. I already knew how to enter an unfamiliar technical domain and work out its rules. I had simply lost sight of that while exhausted.

Results:

- I stayed.
- Over the following years, I became one of the engineers building House Numbers' production LLM and agentic systems, with work ranging from the Communication Service to evaluation infrastructure and operational tooling.
- Areas that had initially made me question my place became areas where teammates regularly trusted my judgment.

Owning a critical communications service end to end

Situation: At House Numbers, loan processors and support staff communicated with the same borrowers and external contacts across different inboxes and phone numbers. A shared inbox centralized email, but operations still had to manually determine which loan and team member owned each communication.

Task: Leadership identified the need to automate that process and assigned the technical problem to me.

Actions:

- My ownership began with studying how operations performed triage and researching the integration capabilities of Front.
- I implemented webhook ingestion, then built LLM-assisted extraction and classification based on the criteria operations used manually.
- Operations and I worked as a feedback loop: I documented and implemented my understanding of their process, and they reported missing cases or incorrect behavior.
- I expanded the architecture to communications arriving through Quo, including calls and text messages. The service analyzes incoming communications, extracts relevant information, matches them to encrypted loan data, and applies deterministic rules to update assignments, tags, and workflow context.
- Once adopted, the service became critical to operations, so I added monitoring, alerts, durable retries, periodic reconciliation for missed webhooks, and diagnostic tooling built on traces, structured logs, and persisted processing history.

Results:

- Other engineers have contributed, but I remain the person with the deepest knowledge of its operational cases and normally review significant changes.
- Human review remains available for exceptions, but operations no longer needs the old manual-triage shifts and usually reports unusual failures to engineering instead.
- A recent seven-month production snapshot contained more than 57,000 communications, with approximately 70% reaching an observed effective-triage outcome. That figure is not LLM accuracy: the remaining bucket includes spam, unsupported edge cases, and an identified observability gap.

Standardizing production debugging across service boundaries

Situation: When House Numbers introduced an on-call rotation, production debugging had no shared investigation method. Each engineer had developed a different way to trace incidents through structured logs, database records, trace IDs, and external events. Their knowledge also tended to follow service ownership. Communication Service incidents usually came to me, while problems in other systems went to the engineer most familiar with that area. The team could investigate production failures, but the speed and depth of the initial response depended heavily on which specialist was available.

Task: I wanted a shared investigation method the whole rotation could use, so that the initial response no longer depended on which specialist was available.

Actions:

- I created an on-call reporting command to standardize those investigation paths. It began with the process I used in the services I knew best, then expanded as we applied it to different systems and incidents.
- Each case exposed another gap. Whenever the command missed an identifier, relationship, or evidence source, I folded that lesson back into it so the next engineer began from a better baseline.
- I later converted the command into a reusable skill and created specialist debugging skills for several production flows.
- Then I shared it. On-call engineers adopted the workflow and contributed improvements from the incidents they handled.
- I proposed the next step: connecting the shared workflow to incident alerts, allowing an agent to begin gathering evidence as soon as a human report or automated alert arrived. Another engineer implemented that automation.

Results:

- Scattered service knowledge became a versioned process the whole team could reuse. An engineer could now begin investigating an unfamiliar service without waiting for its usual expert to become available.
- Today, a human report or a New Relic alert can trigger an agent that reviews the available evidence and develops likely causes. When the problem maps to a code change, the agent can prepare a pull request. An engineer validates the root cause and reviews the proposed fix before anything is merged.
- The workflow removes much of the repetitive correlation work from production debugging and gives every on-call engineer an evidence-backed starting point across services. Initial diagnosis is consistent across the rotation. Each incident can give engineers another opportunity to improve the skills.

Replacing unreliable LLM checks with deterministic validation and bounded AI observations

Situation: One of our early LLM-enabled services analyzed structured data extracted from loan documents and generated a variable list of checks intended to alert loan processors to potential problems. After the system had been in production for an extended period, our product manager shadowed the processing team and discovered that nobody was using those checks. Follow-up conversations showed why: their number and wording varied, some were unclear, and others hallucinated information.

Task: I was asked to determine how to make the analysis useful and trustworthy.

Actions:

- I considered two paths: keep the generic LLM checks and invest in representative golden-set evaluations, or replace the operational checks with deterministic code.
- I recommended the deterministic approach, even though the team initially saw moving away from LLM-generated checks as a regression. The extracted fields already had known structures, and modern agentic coding tools had changed the implementation economics: deterministic checks and tests could now be created and maintained much faster.
- I also compared the continuing model, evaluation, and runtime costs of the LLM approach, as well as the privacy controls required to prevent real-document PII from entering evolving golden sets.
- In less than two weeks, I removed the open-ended LLM overview from every supported document type and expanded deterministic coverage across the service.
- I preserved a narrower role for AI: capped, evaluated observations could provide up to three complementary suggestions beyond the authoritative deterministic checks.

Results:

- The product manager reported successful results from follow-up interviews with the processing team.
- The authoritative checks became stable, reproducible, and covered by deterministic tests instead of changing with model output.
- The design removed unnecessary LLM runtime cost and reduced the need to maintain sensitive golden-set data for rules that could be expressed directly in code.
- We retained LLMs where their flexibility added value, but no longer allowed open-ended generations to define the operational checks processors were expected to trust.

Preventing a monolithic AI pipeline from becoming a reliability problem

Situation: While preparing to migrate House Numbers' document-analysis checks from generic LLM output to deterministic rules, I reviewed the architecture that would carry the change. Party detection, document-specific checks, and checks involving multiple documents were all executed through one large orchestration. The pipeline had not caused a production outage. However, I recognized the risk before adding more complexity. The monolith was difficult to test and maintain, and a slow LLM call or transient failure late in the process could delay or repeat work performed earlier.

Task: I had seen similar long-running, LLM-intensive workflows become operationally difficult in other services, so I decided to improve the architecture before attempting the larger migration.

Actions:

- I mapped the workflow and divided it into three independently testable units connected through the message bus: party detection, checks for an individual document, and cross-document checks.
- Each stage persisted its progress before triggering the next one. This allowed a failed stage to be retried independently instead of replaying the full analysis.
- I also designed the writes to tolerate repeated delivery and added focused unit and integration tests around each boundary.
- Manual reanalysis was later moved into the same pipeline instead of maintaining a separate execution path.

Results:

- The redesigned architecture and the subsequent deterministic-check migration both reached production.
- Earlier stages could complete and preserve their results while downstream work continued.
- The smaller boundaries also made it easier to locate failures, maintain the checks, measure each part of the workflow, and optimize individual stages without treating the entire analysis as one opaque process.

Replacing a prompt-management vendor with repository-native LLM infrastructure

Situation: During our early LLM work at House Numbers, we adopted Orq.ai for prompt versioning, execution and cost tracing, and experiments. It was a reasonable decision at the time: the ecosystem was immature, our experience was limited, and we expected the platform to eventually let non-engineers work with prompts. As our systems grew, especially the Communication Service, the platform began creating friction. Deterministic logic lived in our repository while prompts lived in an external dashboard. Engineers still had to investigate and supervise prompt changes, and we built workarounds to publish prompts and run evaluations that the platform could not easily automate within our development workflow.

Task: I initiated a discussion about replacing Orq.ai and carried it forward over several months. The decision belonged to the team, but our CTO was understandably reluctant because we already had a large prompt inventory and a migration could introduce significant risk.

Actions:

- I documented the workarounds we were maintaining, compared capabilities and alternatives, prepared technical demonstrations and effort estimates, and proposed an incremental service-by-service migration instead of a disruptive replacement.
- After the team agreed, I designed the common migration pattern and implemented it first in our Document Data Extractor and Loan Analysis services.
- I moved prompts into version-controlled files, integrated model providers directly, added provider failover, and connected prompt changes to automated evaluations.
- A teammate had previously created cost-tracing functionality for our Agent Service; I generalized that work into shared infrastructure so additional services could report LLM usage and cost consistently.
- Once the foundation was established, other engineers applied the pattern to the Communication and Task services until the live Orq.ai integration was retired.

Results:

- The incremental migration produced no significant interruption or regression.
- Repository-native evaluations also allowed us to adopt newer, less expensive models while verifying that output quality remained acceptable.
- Prompts, implementation logic, tests, and evaluations can now evolve through the same version-control and review process. Developers across the team, including our CTO, confirmed that the resulting workflow was a significant improvement.
- The new structure also enabled our nontechnical product manager to use Claude Code to investigate reported prompt failures and create more precise engineering tickets, while engineers continued to validate the diagnosis and own the technical decisions.
- We also eliminated a recurring platform cost.

Redesigning a public document-upload flow around privacy and traceability

Situation: At House Numbers, borrowers used a public WordPress form to submit sensitive loan documents without creating an account. The downstream email workflow worked well: it received the documents, used an LLM to extract identifying information, and passed the submission into our existing triage process. The problems were in the client-facing WordPress portal. It was unreliable and offered very little traceability, so borrowers could not tell what had happened after submitting their files and our support team had difficulty reconstructing reported failures. Operationally, we estimated that roughly two out of every three submissions ended in a borrower complaint or support escalation.

Task: I proposed replacing the public portal for an extended period. Once the migration was approved, I researched the requirements and designed and implemented the replacement, including its security architecture. We needed to accept files from an unauthenticated public user without exposing borrower information, loan identifiers, tenant details, routing decisions, or other internal platform state.

Actions:

- I designed the public boundary so that access was validated before a request reached internal services. The browser received an opaque session rather than internal loan or tenant identifiers, and client-provided identity information was removed before the request was forwarded.
- I added CAPTCHA verification, scoped and global rate limiting, upload-size and file-count limits, replay protection, and defensive handling for malformed or expired sessions. Public responses were deliberately minimal so they could not reveal whether a loan had been found or which internal processing path had been selected.
- The simplest implementation would have sent every submission through the existing email workflow. However, uploads already had significant volume, and that path incurred an LLM extraction cost for every submission. The new form collected the relevant matching information as structured fields, so using an LLM to extract the same information again would have added unnecessary cost and processing.
- I instead designed a hybrid routing model. When the submitted information contained strong enough signals to associate the documents with a loan, deterministic rules performed the match directly without involving the LLM or Communication Service. When the signals were incomplete or ambiguous, the system preserved the proven email workflow as a fallback so operations could review the submission manually. This reduced unnecessary LLM processing while ensuring that an uncertain match became a human decision rather than a potentially unsafe automated assignment.
- Every session and file submission produced a correlated audit history that followed the request from the public form through routing and document processing. Sensitive fields were encrypted, while structured logs retained identifiers and processing outcomes without unnecessarily copying raw personal information. I also created a diagnostic skill that uses this history to reconstruct an upload's journey across the participating services.

Results:

- The new page began handling many uploads every day, and borrower complaints about failed uploads stopped.
- Internal support reports still occurred, but we could investigate them quickly. In the cases we reviewed, the trace showed that the upload had succeeded and that the files had later been

renamed, archived, or moved within the operational workflow. Instead of guessing whether the original submission had failed, we could show what had happened and direct support to the files.

- I have applied the same principles elsewhere in the platform: tenant-scoping sensitive reads as defense in depth, removing raw PII payloads from logs, and preventing evaluation artifacts containing real document values from entering source control.

Proved that documented extraction capabilities were disabled for the production account

Situation: While migrating House Numbers' document-analysis checks from generic LLM output to deterministic rules, I needed representative extracted payloads for every supported document type so I could build reliable unit tests. The external platform's documentation said it extracted several types that never appeared with extracted fields in our production database.

Task: I had to find where the structured extraction was being lost before encoding any contract into deterministic tests; building them against guessed data would have encoded a false contract into our code.

Actions:

- I assumed the problem was in our code. I traced the data from incoming webhooks through our persistence and transformation layers, looking for a point where we might be dropping attributes. I could not find one.
- When the next webhook arrived for one of the affected types, I inspected the payload at the boundary and confirmed that the provider had returned a classification but no structured extraction.
- I paused the migration. With the suspect payload and end-to-end trace in hand, I gave the evidence to our product manager, who took it to the vendor's account team.
- To keep the migration moving, I built the missing mocks from the provider's published payload examples, preserving the exact structure their documentation said we should expect.

Results:

- After several exchanges, the vendor confirmed that account-side capabilities for multiple documented document types had been disabled. Once they enabled them, new documents began arriving with the expected extracted fields.
- There was still a historical gap. The original documents remained in the provider's system. The structured extraction expected for those earlier submissions could not be regenerated or redelivered. At that time, House Numbers had no way to recreate the same provider-generated output. We accepted that extraction history as unavailable.
- The mocks gave me a grounded contract for the deterministic checks and unit tests while we waited for new production samples. As fresh extractions arrived, our telemetry and tests confirmed that their shape matched both the documented examples and the contract encoded in the mocks.

Tracing a production crash loop to an environment-specific schema failure

Situation: At House Numbers, a loan processor reported that none of the documents they had uploaded were being processed and that many appeared as "Unknown." The service had restarted 96 times in 24 hours. Its normal range was zero to four restarts per day. The immediate trigger was a webhook carrying a document identifier that our database did not contain. Several operations handling that identifier failed in parallel, and the first rejection masked sibling failures that remained unhandled. Those unhandled errors terminated the process, ECS replaced the task, and each restart could interrupt uploads already in progress. That created a feedback loop capable of producing more missing records and more crashes.

Task: I had the deepest familiarity with that document-processing path, so I took ownership of the incident and began tracing it from the production symptoms backward.

Actions:

- I first contained the cascade. A missing document identifier became a recoverable error that was logged and skipped, and parallel work was allowed to settle without leaving unhandled failures. I also added instrumentation around the upload and persistence boundaries so I could see whether the external system had accepted a file and whether our database had recorded the returned identifier.
- That evidence led me back to a recent major-version migration from the previous Zod release to Zod 4. The migration changed how some schemas treated absent values, and this response schema still relied on the earlier behavior. Local development and the production container also resolved different Zod 4.x versions because the production build did not install from a frozen lockfile. A response field accepted as absent locally was rejected in production. The external processor had successfully accepted the documents, but our application swallowed the validation error before saving their identifiers. When later webhooks arrived, the identifiers looked orphaned. Every document uploaded through the affected production build encountered this failure.
- I reproduced the production error against a real failing response, matched its validation output locally, and changed the schema to preserve backward compatibility. I also changed the extraction path so future upload failures would reach the message bus retry and dead-letter mechanisms instead of disappearing inside a catch block. Regression tests covered each valid response shape, the parallel-failure case, and replay behavior. Persistence was made idempotent when a replay returned an identifier the system had already seen.
- After deploying the correction, I identified the full affected window, prepared and executed the reconciliation work, and reprocessed the documents from that window. I then checked the remaining "Unknown" records in the database to distinguish unrecovered incident failures from legitimate cases where the external system had not sent an update or could not classify the document.

Results:

- The cascade was contained and the failing path became observable and retryable instead of terminating the process.
- The documents from the affected window were reprocessed, and the remaining "Unknown" records were checked to separate unrecovered incident failures from legitimate cases.
- The incident caused no permanent data loss.

Xogito Group — Senior Software Development Engineer · Dec 2020 – Mar 2022

Main backend engineer on a cloud-based, multi-tenant SaaS for real-time quality-control inspections across factories, product lots, and distribution points.

- Owned backend architecture and delivery for the multi-tenant inspection platform end to end.
- After the project manager left, became the team's voice with a frustrated client and recovered the account: joined the frontend developer and QA in client meetings to realign requirements, paired quick wins with the core workflow repairs, and rebuilt trust over a few sprints of approved plans.
- Onboarded and mentored new team members on the business domain and technical stack.

Tech: NestJS, TypeScript, PostgreSQL, Docker, GitLab CI, AWS.

Turning a poor onboarding experience into a repeatable framework

Situation: When I joined Xogito as the first engineer assigned to a new project team, there was no established onboarding support. I received a Confluence page and some general company information, but no clear person to guide me. I had to discover the environment myself and find people on Slack who could answer different questions.

Task: I decided future teammates should have a better experience.

Actions:

- I developed an onboarding approach in which I make the first contact, explain the product and technical foundations, and show the newcomer where information lives and who can help with different topics.
- I stay closely involved during their first couple of days, remain readily available for the following weeks, and act as a bridge to the wider team while encouraging them to develop their own voice and independence.
- In one challenging case, a developer I supervised needed both technical support and confidence. Instead of correcting his mistakes for him, I guided him through finding, debugging, and resolving them himself. I also encouraged him to advocate for a more sustainable workload.

Results:

- His confidence, debugging ability, and overall work improved.
- I have personally used this approach with at least 15 people across Xogito and House Numbers.
- House Numbers later adopted and adapted it for different functions, using rotating onboarding buddies so that people who receive support can eventually guide future newcomers.
- The results have been qualitative rather than formally measured: greater autonomy, more focused questions, stronger meeting participation, and a better ability to find answers independently.

Rebuilding client trust through emerging leadership

Situation: At Xogito, I was a senior full-stack developer and the first developer assigned to a large inspection and delivery-tracking platform. After our project manager resigned, I joined a senior frontend developer and QA in product meetings with the client. Those meetings made it clear that the client was deeply frustrated with our progress and that several core workflows required substantial correction to meet their needs.

Task: Because I had the longest history with the product, I gradually became the team's main spokesperson despite having no formal product-management authority. In practice that meant rebuilding alignment with the client while the team corrected the core workflows.

Actions:

- We increased the meeting cadence to rebuild alignment.
- During those conversations, I noticed the client expressing frustration about smaller capabilities that had been overlooked. I identified several as potential quick wins and proposed delivering them alongside the core repairs.
- I suggested a capacity split, while the frontend developer and QA helped refine the requirements and estimates.
- The team agreed on the approach, and the client approved each sprint plan.

Results:

- Trust returned gradually over two or three sprints. As the client saw consistent progress, the deadline pressure eased and they gave us more time to complete the core repairs.
- After further requirements discussions and demos, the client accepted the corrected workflows.
- I continued leading the product conversations for approximately six months.
- Near my departure, I learned that the same client had commissioned additional projects, and one of their principal contacts invited me to become the first member of an internal technology team they were considering building.

Full-stack development of a platform supporting a fiduciary purchasing agent for FF&E/OS&E procurement, for a US client.

- Built and maintained full-stack features across the procurement platform.
- Migrated the client's historical quote data from their legacy system into the rebuilt platform with custom ETL scripts, expert verification, and financial totals aligned to the legacy calculations — number continuity was an explicit client requirement.

Tech: Node.js, React, PostgreSQL, AWS, Docker.

Preserving financial continuity during a legacy SAP migration

Situation: I was the backend engineer on a project rebuilding a procurement platform whose historical quote and operational data lived in a legacy SAP system. The migration needed to preserve more than individual field values: users expected financial calculations in the new platform to remain consistent with the system they already trusted.

Task: I owned the backend migration strategy and implementation.

Actions:

- Before moving any data, I mapped the source records, relationships, calculations, and corresponding target models.
- I then designed and implemented custom ETL scripts to extract, validate, transform, and load the data, with automated tests before cutover.
- We added human verification by experts who understood the legacy system. They detected that, although the visible source values matched, some derived financial totals differed.
- I investigated the extraction, mappings, and calculations and traced the discrepancy to different rounding behavior between the legacy system and JavaScript. I changed the new calculations to reproduce the established legacy rules. The goal was behavioral continuity for users, not proving that one rounding convention was theoretically better.
- For the final migration, operations were frozen during a controlled extraction and load window so the source could not change underneath the process.
- I also created an on-demand reconciliation process and took database snapshots before reconciliation runs, giving us a rollback point if an update produced corrupted or inconsistent data.

Results:

- The migration was reviewed and approved by the legacy-system experts.
- It completed without data loss or corruption, and after the MVP launched, users stopped relying on the legacy SAP system.
- The reconciliation and snapshot strategy remained available for safe post-launch corrections without putting the migrated dataset at unnecessary risk.

Belatrix Software — Senior NodeJS Software Developer · Jul 2018 – Nov 2018

Led architecture design and the development team for a serverless dynamic-document platform for a major supply-chain provider.

- Designed the serverless architecture and led the development team delivering it.
- Orchestrated automated deployments via CloudFormation.

Tech: AWS, Node.js, React, Docker, Serverless (AWS Lambda / CloudFormation).

Owning a destructive deployment mistake and rebuilding trust

Situation: At Belatrix, I held the title of Senior Node.js Developer while acting as the technical lead for a small team of two junior full-stack developers and two QA engineers. I guided the developers, reviewed their work, gathered requirements with the client, and represented the team internally. For local development, we used a script that cleared DynamoDB data so we could begin testing from a clean state. While building our deployment pipeline for a shared development environment, that script was included in the deployment process. I reviewed the change but failed to notice that the script was not restricted to our project's tables. When the pipeline ran, it cleared data across every DynamoDB table in the shared development environment, including tables belonging to other teams. The environment had no backups, so the deleted information could not be restored and those teams had to reconstruct their development data. No production or customer data was affected, but the incident caused substantial disruption inside the company.

Task: I had not written or executed the script, but I had reviewed it and was responsible for the team.

Actions:

- As soon as we understood what had happened, I reported it rather than waiting for someone else to discover the source. I accepted responsibility with the affected teams and company leadership.
- I did not identify or blame the junior developer who had implemented the change; authorship did not remove my responsibility for approving a destructive operation without adequately evaluating its scope.
- Within our project, we restricted the script to an explicit list of authorized DynamoDB resources and added pipeline controls to prevent it from executing again with an unintended scope.
- We followed the new controls, continued delivering successfully, and demonstrated that the incident had been isolated rather than representative of how the team normally worked.

Results:

- The broader organization introduced stricter permissions and isolation across shared development resources. Those organization-wide protections were not my implementation, and our team initially operated with restricted access and additional supervision.
- We rebuilt trust gradually. After approximately a month, our normal access was restored to the same level as the rest of the company.

Rokk3r — Senior Full Stack Developer and DevOps · Feb 2016 – Jun 2018

Senior full-stack and DevOps engineer across a portfolio of Rokk3r client projects, including a real-time procurement and productivity platform for telecom clients — an engagement his employer Kubesoft assigned him to, covering this whole period rather than a second parallel job.

- Migrated a real-time telecom procurement monolith from self-hosted servers to an AWS Kubernetes microservices cluster, researching the approach and moving large parts of the monolith personally, as part of a company-wide push to adopt microservices across several client monoliths.
- Migrated a mobile app's backend off Parse, a backend-as-a-service being shut down, onto AWS EC2 against the platform's fixed shutdown deadline — reading the platform's own source to work out its poorly documented internals and why the self-hosted version behaved differently from the hosted one.
- Delivered a digital insurance-broker platform and IoT video/academic platforms on experimental hardware.

Tech: Node.js, Docker, Kubernetes, AngularJS, Socket.IO, AWS.

Replacing the rockstar-developer mindset with sustainable performance

Situation: Early in my career, while employed by Kubesoft, I was embedded in a Rokk3r project and working directly with its product manager. It was my first opportunity to contribute to a large project, and I felt that I needed to prove how much I could accomplish. I responded by pushing myself harder instead of communicating. I worked late into the night and treated speed, volume, and personal endurance as evidence that I was a strong developer. The workload was entirely self-imposed; neither the product manager nor the company had asked me to work that way. Eventually, exhaustion began producing the opposite result. My attention to detail deteriorated and the quality of my work declined.

Task: The product manager noticed and invited me to a private conversation. He asked why I was putting myself under pressure that nobody had requested. He reminded me that I already had the team's trust and did not need to prove my value by damaging my health. If I needed rest, assistance, or a deadline adjustment, my responsibility was to communicate that rather than silently trying to absorb everything myself.

Actions:

- That feedback challenged the "rockstar developer" model I had adopted, the idea that the strongest engineer should be able to carry everything alone. I took a week away from the work, recovered, and returned with my quality and working rhythm restored.
- I began communicating capacity and risk more openly, asking for help when necessary, and treating planning conversations as part of engineering rather than as evidence that I could not handle the work.

Results:

- I returned with my quality and working rhythm restored.
- We subsequently delivered the MVP successfully, and the client was very happy with the result.

Kubesoftware SAS — Senior Full Stack & Mobile Developer · Jan 2015 – Jun 2018

Senior full-stack and mobile engineer building web and mobile products for Colombian and international clients; Kubesoftware later assigned him to work directly with Rokk3r, recorded as the separate Rokk3r entry — one continuous employment rather than two parallel jobs.

- Delivered full-stack web and mobile features across Kubesoftware's client products, then spent the rest of the role on the Rokk3r assignment (see that entry) with no separate Kubesoftware client work carried in parallel.
- Built the first backend version of Mutual, a government-supported, hackathon-winning app for parents, creating the delivery momentum that carried it to launch before its proposed deadline.

Tech: Node.js, AngularJS.

Creating momentum when a mission-driven project stalled

Situation: At Kubesoftware, where I was a Senior Full Stack & Mobile Developer, I participated in one of two company teams entering a hackathon. Before competing, we agreed that if either team won, the prize and ownership of the winning project would be shared. The other team won with Mutual, a government-supported application designed to help parents guide children toward activities and conversations that could reduce the risk of drug use. After the win, disagreement emerged over how the prize should be allocated between the participants, product development, and the broader company. The conflict stalled development.

Task: I had not been on the winning team and did not feel entitled to argue over their compensation. What I chose to take on was getting the product built, because I believed in its mission.

Actions:

- I renounced my own share of the prize money and began building the backend.
- After I released the first backend version, other contributors began developing the mobile application and frontend dashboard.
- I worked closely with the mobile developer and continued pushing the project forward through informal leadership.

Results:

- We launched the product before the proposed deadline and handed it over to the government, although we did not receive post-launch usage metrics.
- My contribution created delivery momentum and helped the product launch, but it did not resolve the disagreement that had stalled the team.

Shipping the product while failing to build sustainable ownership

Situation: At Kubesoft, I participated in one of two company teams entering a hackathon. Before the competition, we agreed that if either team won, the prize and ownership of the winning product would be shared. The other team won with Mutual, a government-supported application intended to help parents guide children toward activities and conversations that could reduce the risk of drug use. After the win, disagreements over how the prize should be divided between the participants, product development, and the broader company stalled the project.

Task: I had not been part of the winning team and did not believe I had the right to dispute their compensation. I cared about the product's mission and wanted to create enough momentum for development to begin.

Actions:

- I renounced my share of the prize money and began building the backend.
- After I released the first backend version, other contributors began working on the mobile application and frontend dashboard. I collaborated closely with the mobile developer and continued pushing the work forward through informal leadership.

Results:

- The decision worked from a delivery perspective. We launched the product before the proposed deadline and handed it over to the government.
- However, I do not consider the broader project a success. We delivered the software, but the original conflict remained unresolved and the team never recovered shared ownership.
- My responsibility was not creating the dispute, but choosing how I responded to it. I tried to compensate for the lack of collective commitment with personal effort. That helped us ship, but it did not create a healthy or sustainable team.

Jarvi Games — Full Stack Developer and DevOps · Feb 2013 – Jun 2015

Full-stack and DevOps developer building a real-time game backend with micropayments.

- Built a real-time game backend with micropayments, including the underlying full-stack infrastructure and DevOps.

Tech: Node.js, MongoDB, PostgreSQL, PHP.

Selected Projects

hire-me-mcp — Creator and maintainer · github.com/garasis/hire-me-mcp · hire-me-mcp-web.vercel.app · hire-me-mcp-web.vercel.app/api/mcp

- This portfolio itself: an open-source MCP server, RAG-backed interview agent and Next.js site over one typed career dataset — built as the demonstrable, inspectable evidence of how I ship production software.

cowork — Creator and maintainer · github.com/garasis/cowork

- Open-source Python CLI that orchestrates multiple AI coding agents into role-based teams, with phase gates, per-role evals and cost reporting.

AI Document Extraction Proof of Concept — Lead engineer

- Multi-round proof of concept — never deployed — testing whether vision-LLM document splitting, classification and extraction were a viable replacement candidate for the incumbent OCR vendor on a mortgage platform, with human-reviewed corpora, evaluation and cost tracing, and measurement bugs found and fixed along the way.

LLM Evaluation Infrastructure — Sole engineer / owner

- Solo-built evaluation infrastructure for production LLM features — a labeled corpus, human-graded ground truth, LLM-as-judge scoring, per-call cost tracing and version-controlled prompts.

Monorepo Agent-Consumable Documentation System — Sole engineer / owner

- Solo-built documentation system for a large TypeScript monorepo, with deterministic integrity checks enforced in CI so the docs cannot silently drift from the code they describe.

Skills

- **Expert:** TypeScript, JavaScript, Node.js, React, AWS, Docker, OpenAI API, REST APIs, CI/CD, LLMs (Large Language Models), AI Agents.
- **Proficient:** NestJS, MongoDB, PostgreSQL, Kubernetes, Serverless (AWS Lambda / CloudFormation), Anthropic API, Python, GitLab CI, Microservices, Event-Driven Architecture, Observability, Prompt Engineering, SQL, Turborepo, pnpm, Mentoring & Onboarding, Requirements Gathering, Regulated / Encrypted Data Handling, Vision-based Document Processing.
- **Familiar:** Redis, AngularJS, Socket.IO, PHP, Hybrid Mobile Development.

Education

- Universidad Nacional Abierta y a Distancia (UNAD) — B.S. Systems Engineering (in progress)
- International Scrum Institute — Scrum Master & Product Owner (Jan 2020)